UNIVERSITÉ ALIOUNE DIOP DE BAMBEY

UFR Sciences & Technologies | Master 2 Big Data & Intelligence Artificielle

PROJET TUTORÉ — VERSION 2 CORRIGÉE

Optimisation de la Chaîne Logistique Agricole

Logi-Agri SN

Durée	4 semaines
Équipe	2 étudiants
Domaine	Supply Chain Analytics, IoT Streaming & Optimization
Version	v2 — Bugs corrigés, prérequis ajoutés, barème détaillé
Enseignant	Mr Ahmed Ben Sidy Bouya SEYE — Senior Big Data & AI Engineer | Groupe Sonatel

NiFi • Kafka • Spark Streaming • Pandera • HBase • Hive • Random Forest • Airflow • SciPy linprog

1. Prérequis & Installation

> ℹ️ **Configuration minimale**
> RAM : 16 Go (Spark + Kafka + HBase + Airflow simultanés). CPU : 4 cœurs. OS : Ubuntu 22.04 LTS ou macOS 13+. Docker Engine 24.x + Docker Compose 2.x.

1.1 Vérification de l'environnement

📄 **check_env.sh**

```
# Vérifier Java 11
java -version   # Attendu : openjdk version "11.0.x"

# Vérifier Docker
docker --version        # Docker 24.x minimum
docker compose version  # Docker Compose 2.x minimum

# RAM disponible (minimum 14 Go libres)
free -h

# Limites mémoire pour HBase
echo 'vm.max_map_count=262144' | sudo tee -a /etc/sysctl.conf && sudo sysctl -p
```

1.2 Installation Python

📄 **requirements.txt**

```
# requirements.txt — Logi-Agri SN
# Testé avec Python 3.9 / Java 11 / Spark 3.3

# Core pipeline
pyspark==3.3.2
kafka-python==2.0.2

# Validation
pandera==0.17.2
pandas==2.0.3

# MLOps
apache-airflow==2.7.3
mlflow==2.8.1

# Stockage
pyhive==0.7.0
thrift==0.16.0
happybase==1.2.0        # Client HBase Thrift

# Optimisation & Analyse
scipy==1.11.4
numpy==1.24.4
matplotlib==3.7.3
seaborn==0.12.2

# Installation :
# pip install -r requirements.txt
# Java 11 requis : sudo apt install openjdk-11-jdk
```

📄 **install.sh**

```
python3.9 -m venv venv_logi && source venv_logi/bin/activate
pip install -r requirements.txt
python -c "import pyspark; print(pyspark.__version__)"  # Attendu : 3.3.2
```

1.3 Démarrage de l'infrastructure

start.sh

```
# Cloner le projet
git clone https://github.com/uadb/logi-agri-sn.git && cd logi-agri-sn
mkdir -p data/capteurs data/erp nifi_templates dags models

# Démarrage ordonné
docker compose up -d zookeeper && sleep 10
docker compose up -d kafka nifi hbase hive-metastore && sleep 30
docker compose up -d spark-master spark-worker airflow

# Vérifier les services
docker compose ps

# Initialiser HBase (une seule fois)
python hbase_setup.py

# Accès web :
#   NiFi    : http://localhost:8081
#   Spark   : http://localhost:8080
#   HBase   : http://localhost:16010
#   Airflow : http://localhost:8082
```

2. Codes Sources

2.1 Initialisation HBase — hbase_setup.py

> ⚠️ **Nouveau fichier**
> Sans ce fichier, le pipeline Spark ne peut pas écrire dans HBase. Crée les tables logi:transports, logi:alertes et logi:stocks avec leurs familles de colonnes.

📄 **hbase_setup.py**

```
# hbase_setup.py — Création des tables HBase pour Logi-Agri SN
# NOUVEAU : exécuter une seule fois après docker compose up

import happybase
import logging

logger = logging.getLogger('HBaseSetup')

def create_logi_agri_tables():
    conn = happybase.Connection('hbase', port=9090, timeout=10000)
    conn.open()

    tables_a_creer = {
        # Flux temps réel des transports en cours
        b'logi:transports': {
            b'meta':     {'max_versions': 1},   # voyage_id, produit, zone
            b'kpi':      {'max_versions': 5},   # cout_tonne, taux_perte, statut
            b'position':{'max_versions': 10},  # lat, lon, temp_actuelle
        },
        # Alertes de pertes imminentes (TTL 48h)
        b'logi:alertes': {
            b'alerte':  {'max_versions': 1, 'time_to_live': 172800},
        },
        # Stock entrepôts temps réel
        b'logi:stocks': {
            b'inventaire': {'max_versions': 5},
            b'conditions': {'max_versions': 10},  # temp, humidite
        },
    }

    existantes = [t.decode() for t in conn.tables()]
    for nom_bytes, families in tables_a_creer.items():
        nom = nom_bytes.decode()
        if nom in existantes:
            logger.info(f'Table {nom} déjà existante')
        else:
            conn.create_table(nom_bytes, families)
            logger.info(f'Table {nom} créée')
    conn.close()

if __name__ == '__main__':
    logging.basicConfig(level=logging.INFO)
    create_logi_agri_tables()
```

2.2 Contrats Pandera — schema.py

📄 **schema.py**

```
# schema.py — Contrats Pandera pour Logi-Agri SN

import pandera as pa
from pandera.typing import Series
import pandas as pd, logging
```

```
logger = logging.getLogger('LogiAgriSchema')

class TransportLogisticsSchema(pa.SchemaModel):
    voyage_id:       Series[str]   = pa.Field(unique=True)
    produit:         Series[str]   = pa.Field(isin=['ARACHIDE','MANGUE','RIZ','TOMATE'])
    poids_kg:        Series[float] = pa.Field(gt=0, le=30000)
    temp_moyenne_c:  Series[float] = pa.Field(ge=2.0, le=45.0)
    delai_prevu_h:   Series[int]   = pa.Field(ge=1, le=168)
    date_depart:     Series[str]   = pa.Field(str_matches=r'^\d{4}-\d{2}-\d{2}$')
    zone_origine:    Series[str]   = pa.Field(isin=['CASAMANCE','BASSIN_ARACHIDIER',
                                                    'SINE_SALOUM','NIAYES'])
    zone_dest:       Series[str]   = pa.Field(isin=['DAKAR','EXPORT','THIES','KAOLACK'])
    distance_km:     Series[float] = pa.Field(gt=0, le=800)
    class Config:
        strict = True
        coerce = True

class StockEntrepotSchema(pa.SchemaModel):
    entrepot_id:       Series[str]   = pa.Field()
    produit:           Series[str]   = pa.Field(isin=['ARACHIDE','MANGUE','RIZ','TOMATE'])
    quantite_kg:       Series[float] = pa.Field(ge=0)
    temp_entrepot_c:   Series[float] = pa.Field(ge=0.0, le=40.0)
    humidite_pct:      Series[float] = pa.Field(ge=0.0, le=100.0)
    date_entree:       Series[str]   = pa.Field(str_matches=r'^\d{4}-\d{2}-\d{2}$')
    class Config:
        strict = True
        coerce = True

class CapteurIoTSchema(pa.SchemaModel):
    capteur_id:    Series[str]   = pa.Field()
    voyage_id:     Series[str]   = pa.Field()
    timestamp_utc: Series[str]   = pa.Field(str_matches=r'^\d{4}-\d{2}-
\d{2}T\d{2}:\d{2}:\d{2}$')
    temperature_c: Series[float] = pa.Field(ge=-5.0, le=50.0)
    latitude:      Series[float] = pa.Field(ge=12.0, le=15.5)
    longitude:     Series[float] = pa.Field(ge=-17.5, le=-11.5)
    class Config:
        strict = True
        coerce = True

def validate_and_filter(df, schema_class):
    try:
        return schema_class.validate(df, lazy=True)
    except pa.errors.SchemaErrors as exc:
        err = exc.failure_cases
        logger.warning(f'[{schema_class.__name__}] {len(err)} erreur(s) rejetées')
        valid_idx = df.index.difference(err['index'].dropna().astype(int))
        return df.loc[valid_idx]
```

2.3 Simulateur IoT/ERP — kafka_producer_logi.py

> La fonction compute_pct_perte() calcule maintenant un pct_perte_reel réaliste basé sur le dépassement de température et la durée de transport. Ce champ est la variable cible du modèle Random Forest — il était absent dans la v1, rendant l'entraînement ML impossible.

kafka_producer_logi.py

```
# kafka_producer_logi.py — Simulateur IoT/ERP + pct_perte_reel (CORRIGÉ)
# CORRECTION : génère maintenant pct_perte_reel pour la table historique ML

from kafka import KafkaProducer
import json, random, time, uuid
from datetime import datetime, date, timedelta
```

```
PRODUITS    = ['ARACHIDE','MANGUE','RIZ','TOMATE']
ZONES_PROD  = ['CASAMANCE','BASSIN_ARACHIDIER','SINE_SALOUM','NIAYES']
ZONES_DEST  = ['DAKAR','EXPORT','THIES','KAOLACK']

# Paramètres de conservation par produit
PARAMS = {
    'MANGUE':   {'temp_max':12.0, 'duree_max_h':72,   'perte_base':0.05},
    'ARACHIDE': {'temp_max':30.0, 'duree_max_h':720,  'perte_base':0.02},
    'RIZ':      {'temp_max':35.0, 'duree_max_h':4320, 'perte_base':0.01},
    'TOMATE':   {'temp_max':10.0, 'duree_max_h':48,   'perte_base':0.08},
}

DISTANCES = {
    ('CASAMANCE','DAKAR'):490, ('CASAMANCE','EXPORT'):520,
    ('BASSIN_ARACHIDIER','DAKAR'):190, ('BASSIN_ARACHIDIER','KAOLACK'):50,
    ('SINE_SALOUM','DAKAR'):280, ('NIAYES','DAKAR'):45,
}

producer = KafkaProducer(
    bootstrap_servers=['kafka:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

# CORRECTION : calcul réaliste du pct_perte_reel
# basé sur le dépassement de température et la durée de transport
def compute_pct_perte(produit: str, temp_c: float, delai_h: int) -> float:
    p = PARAMS[produit]
    depassement = max(0.0, temp_c - p['temp_max'])
    facteur_temp  = depassement / 20.0        # 20°C dépassement = +100% perte
    facteur_duree = delai_h / p['duree_max_h']
    perte = p['perte_base'] + facteur_temp * 0.3 + facteur_duree * 0.2
    perte += random.gauss(0, 0.02)             # Bruit réaliste
    return round(max(0.0, min(1.0, perte)), 4)

def gen_transport() -> dict:
    produit   = random.choice(PRODUITS)
    orig      = random.choice(ZONES_PROD)
    dest      = random.choice(ZONES_DEST)
    dist      = DISTANCES.get((orig,dest), random.randint(80,600))
    delai     = max(1, int(dist / random.uniform(40,80)))
    temp      = round(random.uniform(18, 40), 1)
    perte     = compute_pct_perte(produit, temp, delai)
    return {
        'raw_entity_id':  f'T_{uuid.uuid4().hex[:8].upper()}',
        'voyage_id':      f'VY{random.randint(10000,99999)}',
        'produit':        produit,
        'poids_kg':       round(random.uniform(500, 15000), 1),
        'temp_moyenne_c': temp,
        'temp_max_tolere':PARAMS[produit]['temp_max'],
        'delai_prevu_h':  delai,
        'date_depart':    str(date.today()),
        'zone_origine':   orig,
        'zone_dest':      dest,
        'distance_km':    float(dist),
        'cout_fcfa_km':   round(random.uniform(250, 600), 1),
        # CORRECTION : pct_perte_reel maintenant généré ici
        # Dans la réalité, cette valeur vient du constat à la livraison
        # Ici simulée pour alimenter la table historique ML
        'pct_perte_reel': perte,
        'statut':         'EN_COURS',
    }

def gen_capteur(voyage_id: str) -> dict:
    return {
        'capteur_id':    f'CPT{random.randint(1000,9999)}',
        'voyage_id':     voyage_id,
        'timestamp_utc': datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%S'),
        'temperature_c': round(random.uniform(10, 42), 1),
        'latitude':      round(random.uniform(12.3, 15.2), 4),
        'longitude':     round(random.uniform(-17.2, -11.8), 4),
    }

if __name__ == '__main__':
```

```
    print('Simulateur Logi-Agri SN démarré...')
    while True:
        evt = gen_transport()
        producer.send('logi_raw', evt)
        print(f'→ {evt["produit"]} | {evt["zone_origine"]}→{evt["zone_dest"]}
              | perte={evt["pct_perte_reel"]:.2%}')
        for _ in range(3):
            producer.send('logi_capteurs', gen_capteur(evt['voyage_id']))
        producer.flush()
        time.sleep(3)
```

2.4 Pipeline Spark Streaming — streaming_logistics.py

1) La jointure stream-stream (transport + capteurs) est remplacée par 2 queries indépendants — la jointure est déplacée dans Hive/Airflow. 2) COALESCE protège chaque champ nullable. 3) poids_kg_safe évite la division par zéro sur le calcul coût/tonne.

streaming_logistics.py

```
# streaming_logistics.py — Pipeline Spark Streaming Logi-Agri SN (CORRIGÉ)
# CORRECTIONS :
#   1. temp_agg (capteurs) est maintenant joint au pipeline principal
#   2. COALESCE sur tous les champs nullables après jointure LEFT
#   3. Division par zero protégée sur poids_kg

import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, sha2, concat, lit, from_json, to_json, struct,
    window, avg, max as spark_max, when, current_timestamp,
    round as spark_round, coalesce
)
from pyspark.sql.types import StructType, StructField, StringType, FloatType, IntegerType
from pyspark.ml import PipelineModel

SALT      = os.environ.get('LOGI_SECRET_SALT', 'default_salt')
BROKERS   = 'kafka:9092'
MODEL_PATH= 'hdfs:///logi_agri/models/perte_model_latest'

spark = SparkSession.builder \
    .appName('LogiAgri_SN_Streaming') \
    .config('spark.sql.shuffle.partitions', '4') \
    .config('spark.jars.packages',
            'org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.0') \
    .getOrCreate()
spark.sparkContext.setLogLevel('WARN')

transport_schema = StructType([
    StructField('raw_entity_id',   StringType(),  True),
    StructField('voyage_id',       StringType(),  True),
    StructField('produit',         StringType(),  True),
    StructField('poids_kg',        FloatType(),   True),
    StructField('temp_moyenne_c',  FloatType(),   True),
    StructField('temp_max_tolere', FloatType(),   True),
    StructField('delai_prevu_h',   IntegerType(), True),
    StructField('date_depart',     StringType(),  True),
    StructField('zone_origine',    StringType(),  True),
    StructField('zone_dest',       StringType(),  True),
    StructField('distance_km',     FloatType(),   True),
    StructField('cout_fcfa_km',    FloatType(),   True),
    StructField('pct_perte_reel',  FloatType(),   True),
])

capteur_schema = StructType([
    StructField('capteur_id',    StringType(), True),
    StructField('voyage_id',     StringType(), True),
    StructField('timestamp_utc', StringType(), True),
    StructField('temperature_c', FloatType(),  True),
    StructField('latitude',      FloatType(),  True),
    StructField('longitude',     FloatType(),  True),
])

# Table de référence coûts (broadcast statique — PAS un stream)
cout_ref_data = [
    ('CASAMANCE','DAKAR',490,450.0), ('CASAMANCE','EXPORT',520,480.0),
    ('BASSIN_ARACHIDIER','DAKAR',190,300.0), ('BASSIN_ARACHIDIER','KAOLACK',50,250.0),
    ('SINE_SALOUM','DAKAR',280,350.0), ('NIAYES','DAKAR',45,280.0),
]
cout_ref = spark.createDataFrame(
    cout_ref_data, ['zone_origine','zone_dest','dist_ref_km','cout_ref_fcfa_km']
)
```

```
def read_kafka(topic, schema):
    return (spark.readStream
        .format('kafka')
        .option('kafka.bootstrap.servers', BROKERS)
        .option('subscribe', topic)
        .option('startingOffsets', 'latest')
        .load()
        .select(from_json(col('value').cast('string'), schema).alias('d'))
        .select('d.*')
        .withColumn('event_ts', current_timestamp())
    )

transport_df = read_kafka('logi_raw',      transport_schema)
capteur_df   = read_kafka('logi_capteurs', capteur_schema)

# ── Privacy Layer ──────────────────────────────────────────────────────
secure_df = (transport_df
    .withColumn('entity_id_secure',
                sha2(concat(col('raw_entity_id'), lit(SALT)), 256))
    .drop('raw_entity_id')
)

# ── CORRECTION : agrégation capteurs puis jointure stream + static ────────
# On agrège les capteurs sur une fenêtre et on garde le résultat en mémoire.
# La jointure stream-stream est remplacée par un writeStream + lecture Hive
# pour éviter les contraintes de watermark croisé.
#
# Architecture correcte :
#   Stream transport  → KPIs + scoring → écriture Kafka/HBase
#   Stream capteurs   → agrégation temp → écriture table Hive temp_agg
#   DAG Airflow       → jointure Hive transport + temp_agg pour ML

# ── KPI 1 : Coût logistique (jointure stream + static) ────────────────────
kpi_cout = (secure_df
    .join(cout_ref, on=['zone_origine','zone_dest'], how='left')
    .withColumn('cout_fcfa_km_eff',
                coalesce(col('cout_fcfa_km'), col('cout_ref_fcfa_km'), lit(400.0)))
    .withColumn('cout_total_fcfa',
                spark_round(col('distance_km') * col('cout_fcfa_km_eff'), 0))
    .withColumn('poids_kg_safe',
                coalesce(col('poids_kg'), lit(1.0)))  # Protection division /0
    .withColumn('cout_par_tonne_fcfa',
                spark_round(col('cout_total_fcfa') / (col('poids_kg_safe')/1000.0), 0))
)

# ── KPI 2 : Scoring risque de perte ────────────────────────────────────────
enriched = (kpi_cout
    .withColumn('depassement_temp',
                when(col('temp_moyenne_c') > col('temp_max_tolere'),
                     col('temp_moyenne_c') - col('temp_max_tolere')).otherwise(0.0))
    .withColumn('risque_perte_score',
                spark_round(
                    coalesce(col('pct_perte_reel'),
                             (col('depassement_temp')/10.0) +
                             (col('delai_prevu_h')/100.0)),
                    3))
    .withColumn('statut_alerte',
                when(col('risque_perte_score') > 0.4, lit('CRITIQUE'))
                .when(col('risque_perte_score') > 0.2, lit('ATTENTION'))
                .otherwise(lit('NORMAL')))
)

# ── Chargement modèle ML (conditionnel) ────────────────────────────────────
try:
    model = PipelineModel.load(MODEL_PATH)
    enriched = model.transform(enriched)
    print('[ML] Modèle chargé depuis HDFS')
except Exception as e:
    print(f'[ML] Modèle absent ({e}) — scoring par règles expertes')

# ── Écriture streams ───────────────────────────────────────────────────────
```

```
q1 = (enriched
    .select(to_json(struct('*')).alias('value'))
    .writeStream.format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('topic', 'logi_alertes')
    .option('checkpointLocation', '/tmp/logi_transport_ckpt')
    .outputMode('append').start()
)

# ── Agrégation capteurs IoT (stream indépendant) ──────────────────────────
q2 = (capteur_df
    .withWatermark('event_ts', '30 minutes')
    .groupBy(window('event_ts','1 hour'), 'voyage_id')
    .agg(
        spark_max('temperature_c').alias('temp_max_reelle'),
        avg('temperature_c').alias('temp_moy_reelle')
    )
    .select(to_json(struct('*')).alias('value'))
    .writeStream.format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('topic', 'logi_temp_agg')
    .option('checkpointLocation', '/tmp/logi_capteurs_ckpt')
    .outputMode('update').start()
)

q1.awaitTermination()
```

2.5 Entraînement ML — train_perte_model.py

Maintenant que le simulateur génère pct_perte_reel, la table historique_transport est alimentée correctement. Le modèle peut s'entraîner sur des données réelles.

train_perte_model.py

```
# train_perte_model.py — Random Forest sur historique Hive

from pyspark.sql import SparkSession
from pyspark.ml.feature    import VectorAssembler, StringIndexer, StandardScaler
from pyspark.ml.regression import RandomForestRegressor
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml            import Pipeline
import argparse

parser = argparse.ArgumentParser()
parser.add_argument('--output-path',
default='hdfs:///logi_agri/models/perte_model_latest')
parser.add_argument('--window-days', type=int, default=90)
args = parser.parse_args()

spark = SparkSession.builder.appName('LogiAgri_Train') \
    .enableHiveSupport().getOrCreate()

# CORRECTION : pct_perte_reel est maintenant présent grâce au simulateur v2
df = spark.sql(f"""
    SELECT temp_moyenne_c, temp_max_tolere,
           (temp_moyenne_c - temp_max_tolere) AS depassement_temp,
           delai_prevu_h, distance_km, produit, poids_kg,
           pct_perte_reel
    FROM logi_agri.historique_transport
    WHERE date_depart >= DATE_SUB(CURRENT_DATE, {args.window_days})
      AND pct_perte_reel IS NOT NULL
""").na.drop()

train, test = df.randomSplit([0.8, 0.2], seed=42)
print(f'Train={train.count()} | Test={test.count()}')

pipeline = Pipeline(stages=[
    StringIndexer(inputCol='produit', outputCol='produit_idx'),
    VectorAssembler(
        inputCols=['temp_moyenne_c','depassement_temp','delai_prevu_h',
                   'distance_km','poids_kg','produit_idx'],
        outputCol='features_raw'
    ),
    StandardScaler(inputCol='features_raw', outputCol='features',
                   withStd=True, withMean=True),
    RandomForestRegressor(featuresCol='features', labelCol='pct_perte_reel',
                          numTrees=100, maxDepth=8, seed=42),
])

model = pipeline.fit(train)
preds = model.transform(test)

ev = RegressionEvaluator(labelCol='pct_perte_reel', predictionCol='prediction')
print(f'RMSE={ev.setMetricName("rmse").evaluate(preds):.4f}')
print(f'R²  ={ev.setMetricName("r2").evaluate(preds):.4f}')

model.write().overwrite().save(args.output_path)
print(f'Modèle sauvegardé : {args.output_path}')
spark.stop()
```

2.6 DAG Airflow — logi_agri_retrain_dag.py

dags/logi_agri_retrain_dag.py

```
# dags/logi_agri_retrain_dag.py — DAG Airflow Logi-Agri SN

from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.operators.dummy  import DummyOperator
from airflow.utils.dates      import days_ago
from datetime import timedelta
import subprocess, logging

logger = logging.getLogger('logi_agri_dag')
default_args = {'owner':'seye_ahmed','retries':2,'retry_delay':timedelta(minutes=5),
                'email_on_failure':True,'email':['logi-agri@sonatel.sn']}

def aggregate_transports(**ctx):
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore', port=10000)
    cursor = conn.cursor()
    cursor.execute("""
        INSERT INTO logi_agri.historique_transport
        SELECT voyage_id, produit, poids_kg, temp_moyenne_c, temp_max_tolere,
               delai_prevu_h, distance_km, cout_total_fcfa, cout_par_tonne_fcfa,
               pct_perte_reel, date_depart
        FROM logi_agri.flux_temps_reel
        WHERE statut = 'TERMINE'
          AND date_depart = DATE_SUB(CURRENT_DATE, 1)
    """)
    logger.info('Agrégation transports terminée')

def calculate_kpis(**ctx):
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore', port=10000)
    cursor = conn.cursor()
    cursor.execute("""
        INSERT OVERWRITE TABLE logi_agri.kpi_logistique
        SELECT produit, zone_origine, zone_dest,
               COUNT(*)                        AS nb_voyages,
               AVG(cout_par_tonne_fcfa)        AS cout_moy_tonne_fcfa,
               AVG(pct_perte_reel) * 100       AS taux_perte_moyen_pct,
               SUM(poids_kg) / 1000.0          AS tonnage_total,
               AVG(delai_prevu_h)              AS delai_moyen_h,
               CURRENT_DATE                    AS date_calcul
        FROM logi_agri.historique_transport
        WHERE date_depart >= DATE_SUB(CURRENT_DATE, 30)
        GROUP BY produit, zone_origine, zone_dest
    """)
    logger.info('KPIs recalculés')

def check_deviation(**ctx):
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore', port=10000)
    cursor = conn.cursor()
    cursor.execute("""
        SELECT COALESCE(AVG(ABS(risque_perte_score - pct_perte_reel)), 0.0)
        FROM logi_agri.flux_temps_reel
        WHERE date_depart >= DATE_SUB(CURRENT_DATE, 7)
          AND pct_perte_reel IS NOT NULL
    """)
    dev = cursor.fetchone()[0] or 0.0
    logger.info(f'Déviation modèle : {dev:.4f}')
    ctx['ti'].xcom_push(key='deviation', value=dev)
    return 'retrain_model' if dev > 0.05 else 'skip_retrain'

def retrain_model(**ctx):
    result = subprocess.run([
        'spark-submit', '--master', 'spark://spark-master:7077',
        '/opt/airflow/models/train_perte_model.py',
        '--output-path', 'hdfs:///logi_agri/models/perte_model_latest',
        '--window-days', '90'
    ], capture_output=True, text=True, timeout=3600)
    if result.returncode != 0:
        raise Exception(f'Spark job failed:\n{result.stderr}')
```

```
    logger.info('Modèle mis à jour')

with DAG('logi_agri_sn_pipeline', default_args=default_args,
         description='KPI + Réentraînement Logi-Agri SN',
         schedule_interval='0 3 * * *', start_date=days_ago(1),
         catchup=False, tags=['logi-agri','mlops','supply-chain']) as dag:

    start  = DummyOperator(task_id='start')
    skip   = DummyOperator(task_id='skip_retrain')
    end    = DummyOperator(task_id='end')
    t1 = PythonOperator(task_id='aggregate_transports',
                        python_callable=aggregate_transports, provide_context=True)
    t2 = PythonOperator(task_id='calculate_kpis',
                        python_callable=calculate_kpis, provide_context=True)
    branch = BranchPythonOperator(task_id='check_deviation',
                                  python_callable=check_deviation, provide_context=True)
    t3 = PythonOperator(task_id='retrain_model',
                        python_callable=retrain_model, provide_context=True)
    start >> t1 >> t2 >> branch >> [t3, skip]
    t3 >> end; skip >> end
```

2.7 Tables Hive — hive_setup.sql

> Toutes les vues Hive utilisent COALESCE pour les champs nullable. La colonne pct_perte_reel est maintenant présente dans flux_temps_reel et historique_transport.

hive_setup.sql

```
-- hive_setup.sql — Tables et vues Hive Logi-Agri SN (CORRIGÉ)

CREATE DATABASE IF NOT EXISTS logi_agri
    COMMENT 'Logistique agricole Sénégal — UADB 2025';
USE logi_agri;

CREATE TABLE IF NOT EXISTS flux_temps_reel (
    entity_id_secure    STRING, voyage_id STRING, produit STRING,
    poids_kg FLOAT, temp_moyenne_c FLOAT, delai_prevu_h INT,
    zone_origine STRING, zone_dest STRING, distance_km FLOAT,
    cout_total_fcfa FLOAT, cout_par_tonne_fcfa FLOAT,
    risque_perte_score FLOAT, pct_perte_reel FLOAT,
    statut_alerte STRING, statut STRING, ingestion_ts TIMESTAMP
)
PARTITIONED BY (date_depart STRING, produit_part STRING)
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

CREATE TABLE IF NOT EXISTS historique_transport (
    voyage_id STRING, produit STRING, poids_kg FLOAT,
    temp_moyenne_c FLOAT, temp_max_tolere FLOAT,
    delai_prevu_h INT, distance_km FLOAT,
    cout_total_fcfa FLOAT, cout_par_tonne_fcfa FLOAT,
    pct_perte_reel FLOAT COMMENT 'Variable cible ML — maintenant renseignée par simulateur
v2',
    date_depart DATE
) STORED AS ORC;

CREATE TABLE IF NOT EXISTS kpi_logistique (
    produit STRING, zone_origine STRING, zone_dest STRING,
    nb_voyages INT, cout_moy_tonne_fcfa FLOAT, taux_perte_moyen_pct FLOAT,
    tonnage_total FLOAT, delai_moyen_h FLOAT, date_calcul DATE
) STORED AS ORC;

-- CORRECTION : COALESCE null-safe + LEFT JOIN partout
CREATE OR REPLACE VIEW vue_goulots_entrepots AS
SELECT zone_dest AS entrepot, produit,
       COUNT(*) AS nb_voyages,
```

```
       AVG(delai_prevu_h) AS delai_moyen_h,
       SUM(COALESCE(poids_kg,0)) / 1000.0 AS tonnage_t,
       AVG(COALESCE(risque_perte_score,0)) AS risque_moyen,
       CASE WHEN AVG(delai_prevu_h) > 48 THEN 'GOULOT_CRITIQUE'
            WHEN AVG(delai_prevu_h) > 24 THEN 'GOULOT_MODERE'
            ELSE 'NORMAL' END AS statut_goulot
FROM flux_temps_reel
WHERE statut = 'EN_COURS' AND date_depart = CURRENT_DATE()
GROUP BY zone_dest, produit;

CREATE OR REPLACE VIEW vue_pertes_par_region AS
SELECT zone_origine, produit,
       SUM(COALESCE(poids_kg,0) * COALESCE(risque_perte_score,0))/1000.0 AS
tonnes_perdues_est,
       SUM(COALESCE(poids_kg,0)) / 1000.0 AS tonnage_total_t,
       AVG(COALESCE(risque_perte_score,0)) * 100 AS taux_perte_pct,
       SUM(COALESCE(cout_total_fcfa,0) * COALESCE(risque_perte_score,0)) AS
cout_perte_fcfa
FROM flux_temps_reel
WHERE date_depart >= DATE_SUB(CURRENT_DATE(), 30)
GROUP BY zone_origine, produit
ORDER BY tonnes_perdues_est DESC;
```

3. Guide de Test

3.1 Tester Kafka

test_kafka.sh

```
# Topics attendus après docker compose up
docker exec kafka kafka-topics.sh \
  --bootstrap-server localhost:9092 --list
# Attendu : logi_raw, logi_capteurs, logi_alertes, logi_temp_agg

# Démarrer le simulateur
python kafka_producer_logi.py &

# Lire les événements transport (avec pct_perte_reel)
docker exec kafka kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic logi_raw --from-beginning --max-messages 3

# Lire les alertes générées par Spark
docker exec kafka kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic logi_alertes --from-beginning --max-messages 3
# Attendu : JSON avec statut_alerte = CRITIQUE / ATTENTION / NORMAL
```

3.2 Tester HBase

test_hbase.sh

```
# Lister les tables
docker exec hbase hbase shell <<EOF
list
EOF
# Attendu : logi:transports, logi:alertes, logi:stocks

# Vérifier les données en temps réel
docker exec hbase hbase shell <<EOF
scan 'logi:alertes', {LIMIT => 5}
EOF
```

3.3 Tester le Pipeline Spark

test_spark.sh

```
# Lancer le pipeline (les 2 queries s'exécutent en parallèle)
docker exec spark-master spark-submit \
  --master spark://spark-master:7077 \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.0 \
  /opt/streaming_logistics.py

# Générer une alerte CRITIQUE (forcer une MANGUE à 40°C)
# Dans kafka_producer_logi.py, modifier temporairement :
# temp = 40.0  # >> temp_max MANGUE (12°C) → perte > 0.4 → CRITIQUE
```

4. Livrables Attendus

#	Livrable	Contenu attendu
1	**Template NiFi (.xml)**	Flux GetFile → RouteOnAttribute → PublishKafka séparant Production, Stock et Transport
2	**Pipeline Spark**	streaming_logistics.py + hbase_setup.py avec 2 queries indépendants, COALESCE et scoring fonctionnel
3	**Dashboard KPI**	Vue Hive vue_goulots_entrepots + visualisation des routes CRITIQUE/ATTENTION/NORMAL
4	**Rapport d'Optimisation**	Corrélation délai/perte + plan de transport optimal (linprog) + heatmap pertes par région

5. Barème de Notation — /20 points

4 niveaux de maîtrise pour chaque critère. Bonus documentation jusqu'à +2 pts. La démonstration live d'une alerte CRITIQUE est exigée en soutenance.

Critère	Pts	Insuffisant (0)	Passable (50%)	Bien (75%)	Excellent (100%)
Ingestion NiFi Multi-Flux	3 pts	NiFi absent	1 flux sur 3	2 flux, messages partiels	3 flux séparés, messages complets dans logi_raw et logi_capteurs
Validation & Privacy Pandera	4 pts	Pas de validation ni d'anonymisation	1 schéma sur 3 OU anonymisation sans sel	2 schémas + SHA-256 + sel, PII visible en log	3 schémas + SHA-256 + sel + aucun PII dans HBase
Streaming & Jointures KPI	4 pts	Pas de pipeline fonctionnel	Lecture Kafka seule sans KPIs	Coût/tonne calculé, COALESCE absent	2 queries indépendants, COALESCE sur tous les nulls, coût/tonne correct
MLOps & Prédiction + Airflow	5 pts	Pas de modèle ni de DAG	Scoring par règles expertes seul	Random Forest entraîné, DAG sans branchement	Random Forest + DAG avec déviation > 5% + RMSE/R^2 affichés en soutenance
Analytics & Dashboards KPI	4 pts	Ni HBase ni Hive configurés	Tables créées mais vides	Vues Hive fonctionnelles, dashboard statique	Dashboard live + vue_goulots + rapport linprog avec réduction de coût chiffrée
TOTAL	20 pts				

Bonus Documentation (jusqu'à +2 pts)

Élément	Points bonus
README.md avec docker compose up + exemple de sortie	+0.5 pt
Commentaires de code en français (tous scripts)	+0.5 pt
Tests unitaires pytest sur les schémas Pandera	+0.5 pt
Démo live avec alerte CRITIQUE générée sans erreur	+0.5 pt

⚠️ **Exigence démonstration**

Le binôme doit déclencher au moins 1 alerte CRITIQUE en live (Mangue à 40°C, délai > 72h) et montrer le message dans le topic logi_alertes. Sans démonstration fonctionnelle, le critère MLOps est plafonné à 2.5/5.

Bon courage

"Always be a solution, never a problem."

Mr Ahmed Ben Sidy Bouya SEYE | Senior Big Data & AI Engineer | Groupe Sonatel